

Módulo 8

Multi- Agentes

construa um
time de IA
especializado,
coordenado e

economicamente

v2 — 2026 GPT-4o / GPT-4o-mini
inteligente.

Quando um agente vira gargalo?

Sinais de que você precisa de multi-agentes

- O agente **esquece contexto** porque a thread está longa demais
- Você precisa de **múltiplas especialidades** simultâneas
- Crons estão **competindo** pelo mesmo contexto
- O agente faz **muita coisa ao mesmo tempo** e fica lento
- Você quer isolar **domínios diferentes** (conteúdo ≠ ops ≠ código)

Sinais de que ainda é cedo

- Você ainda está **aprendendo** a dar contexto
- O agente faz **1–2 tarefas** por dia
- Você não tem **processos definidos** o suficiente
- A complexidade de gerenciar múltiplos agentes vai **superar o benefício**

⚠ **Regra de ouro:** 2 agentes bem feitos > 6 agentes bagunçados. Multi-agentes sem fundamento = caos. Se ainda está aprendendo a dar contexto, foque no Módulo 4 primeiro.

Arquiteturas de multi-agentes

Hub + Workers (v1)

1 agente principal (Opus/GPT-4o) + workers (GPT-4o-mini). Simples, bom pra começar.

Recomendado iniciante

Boss + Workers (v2)

CoS coordena bosses por domínio. Cada boss gerencia seus workers. Escalável.

Recomendado produção

ACP bind (v2026.4+)

Agentes isolados em topics Telegram. Cada um com contexto próprio, isolado do hub.

Avançado

ACPX Runtime (v2026.4+)

Agentes leves rodando embedded no gateway. Coordenação sem LLM externo.

Experimental

Leveling System — Como os agentes sobem de nível

Todo agente novo começa em **L1 (Observer)**. Promoção é manual e baseada em performance.

L1

OBSERVER

Zero autonomia.
Output sempre
revisado por
você. Cada
entrega é
revisada.

L2

CONTRIBUTOR

Baixa
autonomia. Pode
sugerir, não
executar
sozinho. Review
semanal.

L3

OPERATOR

Média
autonomia.
Executa dentro
de guidelines.
Review semanal.

L4

TRUSTED

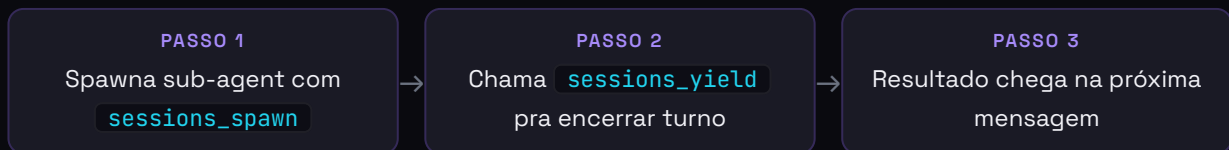
Alta autonomia.
Quase total.
Review
quinzenal.



Regras: Nunca "rushar" um agente pra L3+ sem histórico. Promoção via performance review semanal. Rebaixamento é possível se qualidade cair.

Orquestração: sessions_spawn + sessions_yield

A partir da **v2026.3.13**, o tool `sessions_yield` permite encerrar o turno do agente principal imediatamente após spawnar um sub-agent — sem ficar “preso” esperando.



Antes (sem sessions_yield)

1. Agent spawna sub-agent
 2. Agent continua "rodando" (esperando)
 3. Sub-agent termina → agent continua no mesmo turno
- // Problema: tokens desperdiçados + complexidade

Depois (com sessions_yield) ✓

1. Agent spawna sub-agent
 2. Agent chama sessions_yield → turno encerra
 3. Sub-agent termina
 4. Próximo turno começa com resultado
- // Limpo, econômico, organizado

Como demonstrar ao vivo

Peça ao seu agente: "Spawna um sub-agent com sessions_spawn para uma tarefa simples, usa sessions_yield e me mostra o resultado chegando na próxima mensagem."

sessions_spawn com Attachments (v2026.3.2+)

Você pode enviar arquivos, imagens e buffers diretamente ao spawnar um sub-agent:

```
{
  "sessionTarget": "isolated",
  "payload": {
    "kind": "agentTurn",
    "message": "Analise este relatório e me dê um resumo executivo.",
    "attachments": [
      {
        "type": "file",
        "path": "/workspace/relatorio-mensal.pdf"
      }
    ]
  }
}
```

```
}  
}
```

Casos de uso: passar PDFs pra análise, imagens pra processamento, CSVs pra transformação.

ACP bind — Agentes isolados por topic (v2026.4+)

O ACP bind permite criar agentes **isolados** em topics Telegram específicos — cada um com seu próprio contexto, sem interferir no agente principal.

```
/acp spawn codex --bind here
```

Quando usar ACP bind

- Criar agente de **conteúdo isolado** num topic próprio
- **Experiments** em topics separados
- Ter múltiplos agentes "slave" em **topics diferentes**, cada um com seu contexto

Quando NÃO usar

- Precisa de **contexto compartilhado** com o hub → use `sessions_spawn`
- Precisa de **memória persistente** entre agentes → Boss/Worker é melhor
- Domínios **conectados** entre si

⚠ **ACP bind = isolamento total.** O agente criado não compartilha memória nem contexto com o hub. Use quando precisa de sandbox real.

ACPX Runtime e Embed (v2026.4+)

O **ACPX** é o modelo de runtime que permite a agentes operarem de forma **embedded** (embutida) — rodando como processos leves dentro do gateway, sem overhead de LLM externo para tarefas simples de coordenação.

✓

Benefícios

- Tasks de coordenação sem LLM externo
- Redução de latência
- Redução de custo
- Reação a eventos sem round-trip de API

⚠

Quando considerar

- Orquestração de múltiplos agentes está ficando cara
- Latência entre agentes é um problema
- Você já tem uma arquitetura multi-agente madura

📌 **Recomendação:** A configuração padrão com `sessions_spawn` + `sessions_yield` é suficiente para a maioria dos casos. Explore ACPX quando a orquestração começar a custar caro ou ficar lenta.

Economia de Modelos

USO	MODELO RECOMENDADO	CUSTO RELATIVO
CoS / Hub (coordenação)	GPT-4o	\$\$\$
Bosses (domínio)	GPT-4o	\$\$\$
Crons / automações	GPT-4o-mini	\$
Heartbeats	GPT-4o-mini	\$
Makers (sob demanda)	GPT-4o-mini ou GPT-4o	\$ a \$\$\$
ACP bind agents	GPT-4o	\$\$\$

- Regra #1: Agente que não precisa de GPT-4o → GPT-4o-mini (90% mais barato).
- Regra #2: Heartbeats SEMPRE em GPT-4o-mini.
- Regra #3: Maker idle = \$0 — não existe custo se não é spawnado.

Contexto Compartilhado

Estrutura (v2 real)

```
shared/
├─ decisions.md    ← políticas permanentes
├─ lessons.md      ← aprendizados do time
├─ people.md       ← registry de agentes
├─ tools/          ← integrações compartilhadas
├─ assets/         ← arquivos compartilhados
└─ projects/       ← projetos ativos
```

TEAM.md (legacy)

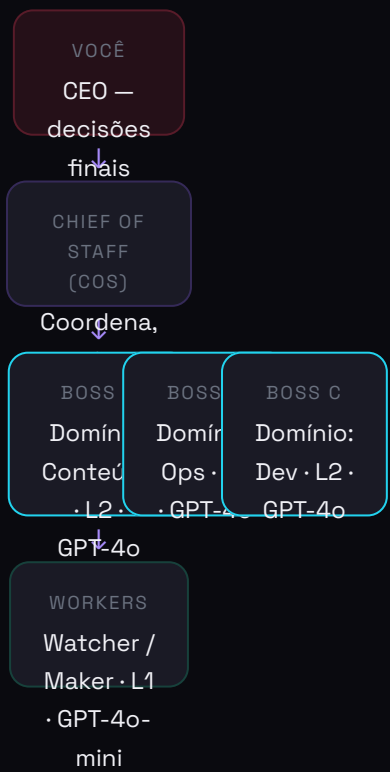
Agente	Papel	Nível	Status
Amora	CoS	L4	Ativo
Boss-A	Domínio A	L2	Ativo
Worker	Função X	L1	Idle

Você (humano) sempre no loop

Regra inegociável: decisões importantes (dinheiro real, mudanças estruturais, deleções) sempre pedem aprovação humana. O agente coordena, você aprova.

Arquitetura v2: Boss + Workers

Quando o Hub vira gargalo, evolua para uma **hierarquia de 3 níveis**: você (CEO) → CoS (Chief of Staff) → Bosses → Workers.



Watcher (fica de plantão)

- Heartbeat ativo (30-60 min)
- Reage a eventos sem o boss pedir
- Custo: ~\$1-2/mês cada

Maker (só existe quando tem trabalho)

- Sem heartbeat — boss usa `sessions_spawn`
- Custo quando idle: **\$0**

”Agents are 30% of the work. The other 70% is the immune system + governance.”

Regra de decisão: Watcher vs Maker

”Esse worker precisa reagir a algo sem que o boss precise pedir?”

- **Sim** → Watcher (heartbeat ativo, custo baixo)
- **Não** → Maker (sem heartbeat, \$0 quando idle)

Checklist — M8 Multi-Agentes

- ☐ Avaliei se multi-agentes faz sentido para meu uso atual
- ☐ Criei 1-2 agentes extras com SOUL.md e AGENTS.md
- ☐ Implementei o sistema de leveling (L1 → L4)
- ☐ Configurei contexto compartilhado (shared/)
- ☐ Defini split de modelos (GPT-4o / GPT-4o-mini)
- ☐ Demonstrei sessions_yield ao vivo
- ☐ Entendi quando usar sessions_spawn vs ACP bind
- ☐ TEOR da economia: Worker idle = \$0 (Maker) | Watcher = ~\$1-2/mês